

JavaScript Key Learnings — Question 2

OJT Assessment Prep — Object.defineProperty, Descriptors & Dynamic DOM

1. Normal Property vs Defined Property

Normally you add properties to objects like this:

```
const obj = {};  
obj.name = "John"; // simple — no control over rules
```

This is quick but gives you **zero control** — anyone can change, delete, or loop over it. For fine-grained control, use **Object.defineProperty()**.

2. Property Descriptors — The 4 Rules

Every JS property has 4 hidden settings called **descriptors**:

Descriptor	What it controls	Default (normal assign)
value	The actual stored value	whatever you set
writable	Can the value be changed?	true
enumerable	Does it show in loops / Object.entries()?	true
configurable	Can it be deleted or redefined?	true

■■ **Important:** When using `defineProperties`, all descriptors default to **false** unless you explicitly set them!

3. Object.defineProperty() Syntax

Define multiple properties with custom rules in one call:

```
Object.defineProperty(adminProfile, {  
  role: {  
    value: "SuperAdmin",  
    writable: false,      // cannot be changed  
    enumerable: true,     // visible in loops  
    configurable: false,  // cannot be deleted/redefined  
  },  
  department: {  
    value: "IT",  
    writable: true,       // can be changed  
    enumerable: false,    // HIDDEN from loops  
    configurable: false,  
  }  
});
```

You can still add normal properties after `defineProperties`:

```
adminProfile.accessLevel = 1; // enumerable: true by default
```

4. Object.entries() — Only Sees Enumerable Properties

Object.entries() returns an array of [key, value] pairs — but only for **enumerable** properties.

```
Object.entries(adminProfile);  
// → [{"role", "SuperAdmin"}, {"accessLevel", 1}]  
//    department is MISSING because enumerable: false
```

■ **This is why** the expected output shows only **role** and **accessLevel** — *department is hidden!*

```
Object.entries(adminProfile).forEach(([key, value]) => {  
  console.log(key, value);  
});
```

5. Dynamic DOM Creation

To create new HTML elements with JS and add them to the page, always follow these 3 steps:

```
// Step 1 – Create the element  
let li = document.createElement("li");  
  
// Step 2 – Set its content  
li.textContent = "Key: role, Value: SuperAdmin";  
  
// Step 3 – Append it to a parent  
ul.appendChild(li);
```

■ ■ The Key Bug — Chaining textContent Assignment

This is a tricky mistake that even experienced devs make:

■ **Wrong** — appends the string, not the element:

```
ul.append(  
  document.createElement("li").textContent = "hello"  
);  
// .textContent = "hello" returns the STRING "hello"  
// so you are appending raw text, not an !
```

■ **Correct** — separate the steps:

```
let li = document.createElement("li"); // store the element  
li.textContent = "hello";               // set text separately  
ul.append(li);                         // append the ELEMENT
```

■ **Rule:** **element.textContent = value** returns the value (string), not the element. Always store the element in a variable first.

■ ■ Full Correct Flow

1. Create empty object: `let adminProfile = {}`
2. `Object.defineProperties()` for role (read-only) and department (hidden)
3. `adminProfile.accessLevel = 1` (normal assignment)
4. Get with `getElementById`
5. `Object.entries(adminProfile)` → only enumerable props

```
6. forEach([key, value]):
  a. let li = document.createElement("li")
  b. li.textContent = `Key: ${key}, Value: ${value}`
  c. ul.append(li)
```

■ Quick Reference — enumerable Cheat Sheet

Property	enumerable	Visible in Object.entries()?
role	true	■ Yes
department	false	■ No — hidden
accessLevel	true	■ Yes